

GymTrack: Vertical Slicing Refactoring Report

Student: Louis Francis Lim

Course: IT342-G01, Systems Integration and Architecture 1

Repository: <https://github.com/louisflim/gymtrack>

Approach: Option B. Refactoring happens directly on `main`, one feature per commit.

What vertical slicing means here

The old layout grouped code by technical layer: everything sat in `controller/`, `service/`, `pages/`, `api/`, regardless of which feature it belonged to. Finding everything related to registration meant hunting across four different folders.

The new layout groups code by feature instead. Each slice owns its own UI, its own API client, and its own backend endpoint and service logic, all in one place:

feature/auth/registration/

Backend: RegistrationController, RegistrationService, RegisterRequest DTO

Web: RegisterPage, RegisterForm, registration/api.js

Mobile: RegisterFragment, RegistrationViewModel, RegistrationRepository

Shared infrastructure, the `User` entity, `UserRepository`, JWT handling, session storage, stays in shared packages for now. It'll get pulled into individual slices as those slices mature.

Commit log

#	Feature	Commit message	Commit hash	Date
---	---------	----------------	-------------	------

1	User registration	`refactor(auth): vertical slice for user registration`	e87ed8d	Jul 7, 2026
2	User login	`refactor(auth): vertical slice for user login`	544bab8	Jul 7, 2026
3	Staff creation	`refactor(auth): vertical slice for admin staff creation`	3407f40	Jul 7, 2026
4	Subscription plans	`refactor(plans): vertical slice for plan management`	672bfbe	Jul 7, 2026
5	Member management	`refactor(members): vertical slice for member management`	887c770	Jul 7, 2026
6	Membership status	`refactor(membership): vertical slice for membership`	8ac7024	Jul 7, 2026
7	Online payment	`refactor(payments): vertical slice for PayMongo checkout`	f7f26f7	Jul 7, 2026
8	QR attendance	`refactor(attendance): vertical slice for QR check-in/out`	7740660	Jul 7, 2026
9	Admin dashboard	`refactor(dashboard): vertical slice for admin KPIs`	61cd3ea	Jul 7, 2026
10	Staff management	`refactor(staff): vertical slice for staff accounts`	65f2c36	Jul 7, 2026

Slice 1: User registration

Gym owners and members both sign up through the same form, and the role they pick determines what gets created behind the scenes (FR-001, FR-002).

BACKEND

File	Responsibility
`feature/auth/registration/RegistrationController.java`	`POST /api/auth/register`
`feature/auth/registration/RegistrationService.java`	Registration business logic
`feature/auth/registration/dto/RegisterRequest.java`	Request payload

WEB

File	Responsibility
`features/auth/registration/RegisterPage.jsx`	Registration screen
`features/auth/registration/RegisterForm.jsx`	Form UI
`features/auth/registration/api.js`	`registerUser()` API call

MOBILE

File	Responsibility
`feature/auth/registration/RegisterFragment.kt`	Registration screen
`feature/auth/registration/RegistrationViewModel.kt`	UI state and validation
`feature/auth/registration/RegistrationRepository.kt`	API call and session save

`POST /api/auth/register` creates an Admin account with a linked gym, or a plain Member account, depending on the role selected. That decision writes to two tables: a new row in `users`, and a new row in `gyms` if the role is ADMIN.

Someone fills out the form on either web or mobile, the client calls the register endpoint, `RegistrationService` checks the role, hashes the password, and creates the gym if needed. The server returns a JWT, the client stores it, and the user lands on their dashboard.

Pulling this out of the horizontal structure meant removing `register()` from `AuthController` and `AuthService`, dropping `registerUser` from `web/src/api/auth.js`, removing `register()` from the mobile `AuthRepository`/`AuthViewModel`, and deleting `web/src/pages/Register/\*` entirely.

TESTING

- Register as a member on web, land on the dashboard
- Register as a gym owner with a gym name, dashboard shows the admin role
- Duplicate email produces an error message
- Same checks pass on the Android app

Slice 2: User login

Anyone with an existing account, Admin, Staff, or Member, signs in with email and password (FR-003).

BACKEND

File	Responsibility
`feature/auth/login/LoginController.java`	`POST /api/auth/login`
`feature/auth/login/LoginService.java`	Authentication and JWT issuance
`feature/auth/login/dto/LoginRequest.java`	Request payload

WEB

File	Responsibility
`features/auth/login/LoginPage.jsx`	Login screen
`features/auth/login/LoginForm.jsx`	Form UI
`features/auth/login/api.js`	`loginUser()` API call

MOBILE

File	Responsibility
`feature/auth/login/LoginFragment.kt`	Login screen
`feature/auth/login/LoginViewModel.kt`	UI state
`feature/auth/login/LoginRepository.kt`	API call and session save

The login endpoint reads from `users` to look up the account by email, nothing gets written here. A user types in their credentials, the client hits `/api/auth/login`, `LoginService` confirms the account is active, runs it through Spring Security, and issues a JWT if everything checks out. The client saves the session and moves to the dashboard.

I removed `login()` from `AuthController`/`AuthService`, dropped `loginUser` from `web/src/api/auth.js`, pulled `login()` out of the mobile `AuthRepository`/`AuthViewModel`, and deleted `web/src/pages/Login/\*`. `AuthViewModel` on mobile now only deals with session state and logout, since the dashboard still needs that.

TESTING

- Valid credentials on web land on the dashboard
- Wrong password shows "Invalid email or password"
- A deactivated account gets a forbidden message
- Same checks pass on Android
- Signing out from the dashboard returns to the login screen

Slice 3: Admin staff creation

Gym owners can create staff accounts tied to their own gym (FR-004).

BACKEND

File	Responsibility
`feature/auth/staff/StaffCreationController.java`	`POST /api/auth/staff`
`feature/auth/staff/StaffCreationService.java`	Staff account creation logic
`feature/auth/staff/dto/CreateStaffRequest.java`	Request payload
`feature/auth/staff/dto/StaffAccountResponse.java`	Created staff summary

WEB

File	Responsibility
`features/auth/staff/CreateStaffForm.jsx`	Staff creation form (admin dashboard)

<code>`features/auth/staff/api.js`</code>	<code>`createStaffUser()`</code> API call
---	---

MOBILE

File	Responsibility
<code>`feature/auth/staff/StaffCreationRepository.kt`</code>	API call for staff creation

``POST /api/auth/staff`` only works for admins, and it writes a new STAFF row to ``users`` with the ``gym_id`` pulled from whoever's making the request.

An admin fills out the staff form from the dashboard, on web or mobile, the client sends the request with a JWT attached, and ``StaffCreationService`` double-checks the caller is actually an admin with a gym before hashing the password and saving. A success message shows up and the staff list refreshes.

This slice finished off the auth feature entirely, so ``AuthController`` and ``AuthService`` are gone now, along with ``web/src/api/auth.js``. ``createStaff()`` came out of the mobile ``AuthRepository``, and ``web/src/components/dashboard/CreateStaffForm.jsx`` got deleted. ``AuthRepository`` on mobile is down to just session and logout duties.

TESTING

- Admin creates staff on web, gets a success message, staff shows up in the staff tab
- Duplicate email produces an error
- A non-admin can't call the endpoint
- Same flow works from the Android admin dashboard

Slice 4: Subscription plans

Gym owners set up plans; members see whatever's active for their own gym when it's time to subscribe (FR-005).

BACKEND

File	Responsibility
<code>`feature/plans/PlanController.java`</code>	<code>`GET/POST/PUT /api/plans`</code>

`feature/plans/PlanService.java`	Plan CRUD scoped to the admin's gym
`feature/plans/dto/PlanRequest.java`	Create/update payload
`feature/plans/dto/PlanResponse.java`	Plan summary

WEB

File	Responsibility
`features/plans/PlanForm.jsx`	Admin plan create/edit form
`features/plans/PlanList.jsx`	Admin plan table
`features/plans/PlanPicker.jsx`	Member plan selection cards
`features/plans/api.js`	Plan API calls

MOBILE

File	Responsibility
`feature/plans/PlanRepository.kt`	Plan API calls

- `GET /api/plans/active`: active plans for a member or staff account's gym
- `GET /api/plans`: every plan, admin only
- `POST /api/plans`: create a plan, admin only
- `PUT /api/plans/{id}`: update a plan, admin only

Everything here lives in `subscription\_plans`, linked back to a gym. An admin creates a plan from the dashboard and it saves against their own gym. Once a member scans the gym QR and gets enrolled, the Plans tab shows them whatever's active. Picking a plan hands off to checkout, which is its own slice.

I deleted `controller/PlanController`, `service/PlanService`, and both plan DTOs from the old locations, along with `web/src/api/plans.js` and the horizontal `PlanForm.jsx`, `PlanList.jsx`, and `member/PlanPicker.jsx`. Plan-related methods came out of the mobile `GymRepository` too.

TESTING

- Admin creates a plan on web, it shows up in the plan list
- Editing a plan saves the changes
- A member sees active plans right after enrolling at a gym

- Same checks pass on Android

Slice 5: Member management

Gym owners can list, search, edit members, and assign them plans (FR-006).

BACKEND

File	Responsibility
`feature/members/MemberController.java`	`GET/PUT /api/members`, `POST /api/members/assign-plan`
`feature/members/MemberService.java`	Member list/update and plan assignment
`feature/members/dto/MemberResponse.java`	Member summary with membership info
`feature/members/dto/MemberUpdateRequest.java`	Profile update payload
`feature/members/dto/AssignPlanRequest.java`	Plan assignment payload

WEB

File	Responsibility
`features/members/MemberTable.jsx`	Member list, edit modal, assign-plan modal
`features/members/api.js`	Member API calls

MOBILE

File	Responsibility
`feature/members/MemberRepository.kt`	Member API calls

- `GET /api/members`: list/search/filter members, admin only
- `PUT /api/members/{id}`: update a member's profile, admin only
- `POST /api/members/assign-plan`: assign a plan to a member, admin only

This one touches two tables: `users` for the profile edits, and `memberships` whenever a plan gets assigned (through `MembershipService`, shared with the membership slice).

An admin opens the Members tab and sees everyone registered at their gym. Editing a member sends a PUT that updates the user row directly. Assigning a plan activates a membership for that member.

I removed ``controller/MemberController``, ``service/MemberService``, and the three member DTOs, along with ``web/src/api/members.js`` and the old ``web/src/components/admin/MemberTable.jsx``. Member-related methods are gone from the mobile ``GymRepository``.

TESTING

- Admin sees the member list on web
- Editing a member's name or email saves correctly
- Assigning a plan to an enrolled member updates their status
- Same checks pass from the Android admin dashboard

Slice 6: Membership status

Tracks where a member stands: subscription status, onboarding progress, gym enrollment (FR-007).

BACKEND

File	Responsibility
<code>`feature/membership/MembershipController.java`</code>	<code>`GET /api/membership/me`</code>
<code>`feature/membership/MembershipService.java`</code>	Status computation, activation, attendance eligibility
<code>`feature/membership/dto/MembershipResponse.java`</code>	Membership and onboarding payload

WEB

File	Responsibility
<code>`features/membership/MembershipCard.jsx`</code>	Member status summary
<code>`features/membership/MemberOnboardingCard.jsx`</code>	Gym enrollment / next-step guidance

<code>`features/membership/api.js`</code>	<code>`fetchMyMembership()`</code>
---	------------------------------------

MOBILE

File	Responsibility
<code>`feature/membership/MembershipRepository.kt`</code>	Membership API call

``GET /api/membership/me`` returns a member's plan, status, gym, and whatever onboarding step comes next. It reads and updates ``memberships`` for status and dates, and touches ``users`` for the gym link and the first-check-in flag.

Opening the dashboard triggers this call. ``MembershipService`` works out the current status, ACTIVE, EXPIRING_SOON, EXPIRED, or NONE, and the UI shows either an onboarding card or a full membership card depending on what step the member is on. Both the payments slice and the admin's assign-plan flow call into the same activation logic here, so status stays consistent no matter how a membership gets activated.

Gone from the old structure: ``controller/MembershipController``, ``service/MembershipService``, ``dto/MembershipResponse``, ``web/src/api/membership.js``, and the horizontal ``MembershipCard.jsx``/``MemberOnboardingCard.jsx``. ``myMembership()`` no longer exists in the mobile ``GymRepository``.

TESTING

- A brand-new member sees the "Scan Gym QR" onboarding step
- Scanning the gym QR advances to the next step
- Buying a plan makes the membership card show the plan and dates
- Same checks pass on Android

Slice 7: Online payment

Members pay for a plan through PayMongo; admins can see the payment history for their gym (FR-008).

BACKEND

File	Responsibility
`feature/payments/PaymentController.java`	Checkout, webhook, list endpoints
`feature/payments/PaymentService.java`	Payment flow and membership activation
`feature/payments/PayMongoService.java`	PayMongo API integration (mocked in dev)
`feature/payments/dto/CheckoutRequest.java`	Checkout payload
`feature/payments/dto/CheckoutResponse.java`	Checkout URL response
`feature/payments/dto/PaymentResponse.java`	Payment record summary

WEB

File	Responsibility
`features/payments/PaymentTable.jsx`	Admin/member payment table
`features/payments/PaymentHistory.jsx`	Member payment history wrapper
`features/payments/PaymentSuccessPage.jsx`	Post-checkout success and mock confirm
`features/payments/PaymentCancelPage.jsx`	Cancelled checkout page
`features/payments/api.js`	Payment API calls

MOBILE

File	Responsibility
`feature/payments/PaymentRepository.kt`	Checkout, history, mock confirm

- `POST /api/payments/checkout`: create a PayMongo session
- `POST /api/payments/webhook`: PayMongo's webhook handler
- `POST /api/payments/confirm-mock`: dev-only mock confirmation
- `GET /api/payments/me`: member's own payment history
- `GET /api/payments`: admin's gym-wide payment list

Every payment record lives in `payments`, along with its status and the matching PayMongo reference.

A member picks a plan, checkout gets created, and they're redirected to PayMongo (or a mock URL locally). Once that succeeds, either through the real webhook or the mock confirm endpoint, the payment flips to PAID and the membership activates. Admins can see every payment for their gym; members only see their own.

This slice removed ``controller/PaymentController``, ``service/PaymentService``, and ``service/PayMongoService`` from their old spots, plus the ``CheckoutRequest``, ``CheckoutResponse``, and ``PaymentResponse`` DTOs. ``web/src/api/payments.js``, the admin ``PaymentTable.jsx``, the member ``PaymentHistory.jsx``, and the ``PaymentSuccess`/`PaymentCancel`` pages are all gone from ``web/src/pages``. Payment methods are out of the mobile ``GymRepository`` too.

TESTING

- Subscribing to a plan redirects to checkout
- A mock payment success activates the membership
- The payment shows up in both the member's history and the admin's payments tab
- Same checks pass on Android

Slice 8: QR attendance

Covers QR-based gym enrollment, member check-in and check-out, and attendance history for both members and admins (FR-006, FR-007).

BACKEND

File	Responsibility
<code>`feature/attendance/AttendanceController.java`</code>	Scan, member logs, gym logs endpoints
<code>`feature/attendance/QrController.java`</code>	Member and gym QR generation
<code>`feature/attendance/AttendanceService.java`</code>	QR generation, scan logic, enrollment
<code>`feature/attendance/dto/ScanRequest.java`</code>	Scan payload
<code>`feature/attendance/dto/AttendanceScanResponse.java`</code>	Staff scan result
<code>`feature/attendance/dto/AttendanceLogResponse.java`</code>	Attendance log summary
<code>`feature/attendance/dto/QrCodeResponse.java`</code>	QR image payload

`feature/attendance/dto/MemberGymScanResponse.java`	Member gym-scan result
---	------------------------

WEB

File	Responsibility
`features/attendance/QrScanner.jsx`	Webcam QR scanner
`features/attendance/AttendanceScannerCard.jsx`	Staff scanner launcher
`features/attendance/GymQrCard.jsx`	Admin/staff gym enrollment QR
`features/attendance/MemberQrCard.jsx`	Member personal QR
`features/attendance/AttendanceHistory.jsx`	Member attendance history
`features/attendance/AttendanceLogTable.jsx`	Admin filtered log table
`features/attendance/AttendanceLogList.jsx`	Shared log list renderer
`features/attendance/api.js`	Attendance and QR API calls

MOBILE

File	Responsibility
`feature/attendance/AttendanceRepository.kt`	QR, scan, and log API calls
`feature/attendance/QrScannerDialogFragment.kt`	Camera QR scanner dialog

- `GET /api/qr/me`: member's personal QR code
- `GET /api/qr/gym`: staff/admin gym enrollment QR
- `POST /api/attendance/scan-gym`: member scans the gym QR to enroll
- `POST /api/attendance/scan`: staff scans a member's QR to check them in or out
- `GET /api/attendance/me`: member's own attendance history
- `GET /api/attendance/gym`: admin/staff gym logs, searchable and filterable by date

`attendance\_logs` holds every check-in and check-out. `users` gets updated for the gym link and the first-check-in flag on enrollment or scan, and `gyms` is where the gym QR payload comes from.

An admin displays the gym's QR code, a member scans it and gets enrolled. From there, staff can open the scanner and scan a member's personal QR to log a check-in or check-out. Members can view their own QR and history; admins filter the gym-wide log.

Removed: ``controller/AttendanceController``, ``controller/QrController``, ``service/QrAttendanceService``, and all five attendance/QR DTOs from their old locations. ``web/src/api/qr.js`` and the horizontal QR/attendance components are gone, and so are mobile's ``data/repository/AttendanceRepository.kt`` and ``ui/fragments/qr/QrScannerDialogFragment.kt``. ``gymAttendance()`` came out of the mobile ``GymRepository``; ``MembershipService`` now imports ``AttendanceService.computeNextStep`` directly.

TESTING

- Scanning the gym QR enrolls a member and updates their onboarding step
- Staff scanning a member QR records a check-in, then a check-out
- Members see their attendance history; admins can filter the gym logs
- Same checks pass on Android

Slice 9: Admin dashboard

Rolls up gym KPIs for the admin's home tab: member counts, subscription status breakdown, today's check-ins, total payments collected.

BACKEND

File	Responsibility
<code>`feature/dashboard/DashboardController.java`</code>	<code>`GET /api/dashboard/stats`</code>
<code>`feature/dashboard/DashboardService.java`</code>	Aggregates gym KPI metrics
<code>`feature/dashboard/dto/DashboardStatsResponse.java`</code>	KPI response payload

WEB

File	Responsibility
<code>`features/dashboard/KpiSummaryGrid.jsx`</code>	Admin KPI card grid

<code>`features/dashboard/api.js`</code>	<code>`fetchDashboardStats()`</code>
--	--------------------------------------

MOBILE

File	Responsibility
<code>`feature/dashboard/DashboardRepository.kt`</code>	Dashboard stats API call

``GET /api/dashboard/stats`` is admin-only and doesn't touch any new tables, it just reads from ``users``, ``memberships``, ``attendance_logs``, and ``payments``.

Opening the Home tab fires that request, ``DashboardService`` counts up members, splits subscriptions into active versus expired, tallies today's check-ins, and totals up revenue. The web ``KpiSummaryGrid`` and the mobile KPI adapter both render whatever comes back.

I deleted ``controller/DashboardController``, ``service/DashboardService``, and ``dto/DashboardStatsResponse`` from the old locations, along with ``web/src/api/dashboard.js`` and ``web/src/components/common/KpiSummaryGrid.jsx``. ``dashboardStats()`` is gone from the mobile ``GymRepository``.

TESTING

- Admin home tab shows all five KPI cards with correct numbers
- Non-admins can't reach the stats endpoint
- Same KPIs show up on the Android admin overview tab

Slice 10: Staff management

Lets gym admins list and update staff accounts, name, email, active status. Creating a staff account still lives in ``feature/auth/staff``, this slice only covers listing and editing.

BACKEND

File	Responsibility
`feature/staff/StaffController.java`	List and update staff endpoints
`feature/staff/StaffService.java`	Staff list/update business logic
`feature/staff/dto/StaffResponse.java`	Staff account summary
`feature/staff/dto/StaffUpdateRequest.java`	Staff update payload

WEB

File	Responsibility
`features/staff/StaffTable.jsx`	Staff list and edit modal
`features/staff/api.js`	`fetchStaff()`, `updateStaff()`

MOBILE

File	Responsibility
`feature/staff/StaffRepository.kt`	Staff list and update API calls

- `GET /api/staff`: list gym staff, admin only
- `PUT /api/staff/{id}`: update a staff account, admin only

`users` rows filtered to role STAFF and the admin's gym cover the data here.

An admin opens the Staff tab, the list loads, and editing a staff member sends a PUT that updates their name, email, or active flag. Creating new staff still goes through `POST /api/auth/staff` from the auth slice, that logic didn't need to move.

I removed `controller/StaffController`, `service/StaffService`, and both staff DTOs from their old spots, along with `web/src/api/staff.js` and `web/src/components/admin/StaffTable.jsx`. Mobile's `GymRepository` was empty after every slice pulled its methods out, so I deleted it and moved the last staff methods into `StaffRepository`. `DashboardViewModel` no longer depends on `GymRepository` either.

TESTING

- Admin Staff tab lists everyone at the gym
- Editing a staff member's name, email, or active status saves correctly

- A duplicate email gets rejected on update
- Same checks pass on Android